

INF560 — DESARROLLO WEB BACKEND

GUÍA 7

CRUD Completo con Formularios

Proyecto acumulativo: **library-app**

CONTENIDO DE LA GUÍA

Esta guía aborda los siguientes temas:

- ▶ Formularios estilizados para creación y edición de libros con Tailwind.
- ▶ Implementación completa de los métodos `store()`, `update()` y `destroy()`.
- ▶ Asociación N:M con autores mediante `sync()` en formularios.
- ▶ CRUD completo de autores con verificación de integridad referencial.
- ▶ Refactorización con `partials _form.blade.php` compartidos.
- ▶ Mensajes flash de éxito y error en el layout principal.
- ▶ Eliminación segura: soft delete con verificación de préstamos activos.

COMPETENCIA

Implementa operaciones CRUD completas en una aplicación Laravel utilizando formularios HTML estilizados, validación inline, asociaciones N:M con `sync()`, eliminación lógica y refactorización mediante `partials`, aplicando buenas prácticas de seguridad (CSRF) e integridad referencial.

Universidad Autónoma Tomás Frías

Carrera de Ingeniería Informática | Potosí, Bolivia

M. Sc. Huáscar Fedor Gonzales Guzmán

1. Objetivos de la Guía

Al concluir esta guía práctica, el estudiante será capaz de:

- Construir formularios HTML completos y semánticamente correctos en Blade, estilizados con utilidades de Tailwind CSS.
- Comprender y aplicar el mecanismo de protección CSRF que ofrece Laravel mediante la directiva `@csrf`.
- Implementar los métodos `store()`, `update()` y `destroy()` de un controlador resource.
- Sincronizar relaciones N:M (libros–autores) usando el método `sync()` de Eloquent en operaciones de creación y actualización.
- Aplicar validación inline con `$request->validate()` y manejar reglas dependientes del registro actual (unique con excepción).
- Implementar eliminación lógica (soft delete) con verificación previa de integridad referencial.
- Refactorizar formularios duplicados extrayendo un partial `_form.blade.php` compartido entre create y edit.
- Mostrar mensajes flash de éxito y error en el layout principal de la aplicación.
- Versionar la entrega con un tag de Git semántico (v0.4.0).

2. Prerrequisitos

Para abordar esta guía con éxito, el proyecto library-app debe encontrarse en el estado resultante de la Guía 6 (tag v0.3.0). En particular, debes contar con:

- Proyecto Laravel 13 con PostgreSQL configurado y migraciones ejecutadas.
- Modelos User, Author, Category, Book, Member y Loan con sus relaciones definidas (incluida la relación N:M libros–autores con tabla pivote `author_book`).
- Factories y seeders cargados con datos de prueba.
- `BookController` y `AuthorController` con los métodos `index()`, `show()` y `create()` ya implementados.
- Layout Blade principal (`layouts/app.blade.php`) con header de navegación y un slot `@yield('content')` o `$slot`.
- Vistas `books/index.blade.php`, `books/show.blade.php` y `books/create.blade.php` existentes (la última con un esqueleto vacío que completaremos en esta guía).
- Tailwind CSS instalado y compilando vía Vite.

NOTA Verificación rápida del estado

Antes de empezar, ejecuta los siguientes comandos para confirmar que tu entorno responde correctamente:

```
// bash
# 1. Verificar la rama y el último tag
git status
git tag --list
```

```
# 2. Levantar el servidor de Laravel
php artisan serve

# 3. En otra terminal, levantar Vite
npm run dev

# 4. Visitar http://127.0.0.1:8000/books y confirmar que se listan libros
```

ADVERTENCIA Si tu proyecto está desactualizado

Si no tienes el tag `v0.3.0`, ejecuta primero `git checkout v0.3.0` y crea una rama de trabajo con `git checkout -b feature/crud-formularios`. Esto te permitirá seguir la guía sin afectar el historial principal hasta que confirmes que todo funciona.

3. Desarrollo Paso a Paso

A lo largo de esta sección construiremos, de manera incremental, todo el ciclo CRUD para libros y autores. Cada bloque de código se entrega completo y funcional para que puedas pegarlo directamente en tu proyecto y ejecutarlo de inmediato.

3.1 Formulario de creación de libros

Comenzamos completando el formulario de alta de libros. Este formulario reúne todos los campos del modelo Book más la asociación N:M con autores. Como la vista `books/create.blade.php` ya existe (con un esqueleto vacío), simplemente reemplazaremos su contenido.

DEFINICIÓN ¿Por qué un formulario tan extenso?

Un libro reúne metadatos descriptivos (título, ISBN, editorial), atributos de catálogo (idioma, año, páginas), recursos visuales (portada) y, sobre todo, asociaciones a otras entidades (categoría y autores). Modelar correctamente este formulario es la base del catálogo de la biblioteca: si un dato falta o queda mal asociado, todas las consultas posteriores quedarán incompletas.

3.1.1 Vista `resources/views/books/create.blade.php`

Reemplaza el contenido completo del archivo por el siguiente código. Comentamos cada bloque importante con un comentario Blade (`{{-- ... --}}`) para que sirva de referencia.

```
// blade
{{-- resources/views/books/create.blade.php --}}
@extends('layouts.app')

@section('title', 'Registrar libro')

@section('content')


{{-- Encabezado --}}
    <div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Registrar libro</h1>
        <a href="{{ route('books.index') }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al
    catálogo</a>
    </div>

    {{-- Bloque global de errores de validación --}}
    @if ($errors->any())
        <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
            <p class="font-semibold text-red-800 mb-2">
                Hay {{ $errors->count() }} error(es) en el formulario:
            </p>
            <ul class="list-disc list-inside text-sm text-red-700">


```

```

        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>
</div>
@endif

<form method="POST" action="{{ route('books.store') }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    {{-- Token CSRF: imprescindible en Laravel para POST/PUT/DELETE --}}
    @csrf

    {{-- Fila 1: título --}}
    <div>
        <label for="title" class="block text-sm font-medium text-slate-700">
            Título <span class="text-red-600">*</span>
        </label>
        <input type="text" name="title" id="title" required
            value="{{ old('title') }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                focus:border-indigo-500 focus:ring-indigo-500">
    </div>

    {{-- Fila 2: ISBN y editorial --}}
    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="isbn" class="block text-sm font-medium text-slate-700">
                ISBN <span class="text-red-600">*</span>
            </label>
            <input type="text" name="isbn" id="isbn" required
                placeholder="978-..."
                value="{{ old('isbn') }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
        <div>
            <label for="publisher" class="block text-sm font-medium text-slate-
700">
                Editorial
            </label>
            <input type="text" name="publisher" id="publisher"
                value="{{ old('publisher') }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
    </div>

    {{-- Fila 3: año, páginas, idioma --}}
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div>
            <label for="publish_year" class="block text-sm font-medium text-
slate-700">
                Año de publicación
            </label>
            <input type="number" name="publish_year" id="publish_year"
                min="1000" max="{{ date('Y') }}"
                value="{{ old('publish_year') }}"

```

```

        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="pages" class="block text-sm font-medium text-slate-700">
            Páginas
        </label>
        <input type="number" name="pages" id="pages" min="1"
            value="{{ old('pages') }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
            focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="language" class="block text-sm font-medium text-slate-
700">
            Idioma
        </label>
        <select name="language" id="language"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
            focus:border-indigo-500 focus:ring-indigo-500">
            @php
                $idiomas = ['Español', 'Inglés', 'Portugués', 'Francés',
'Alemán', 'Otro'];
            @endphp
            @foreach ($idiomas as $idioma)
                <option value="{{ $idioma }}"
                    {{ old('language') == $idioma ? 'selected' : '' }}>
                    {{ $idioma }}
                </option>
            @endforeach
        </select>
    </div>
</div>

{{-- Fila 4: descripción --}}
<div>
    <label for="description" class="block text-sm font-medium text-slate-
700">
        Descripción
    </label>
    <textarea name="description" id="description" rows="4"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">{{
old('description') }}</textarea>
    </div>

{{-- Fila 5: portada y copias totales --}}
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
    <div>
        <label for="cover_url" class="block text-sm font-medium text-slate-
700">
            URL de la portada
        </label>
        <input type="url" name="cover_url" id="cover_url"
            value="{{ old('cover_url') }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm

```

```

        focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="total_copies" class="block text-sm font-medium text-
slate-700">
            Copias totales <span class="text-red-600">*</span>
        </label>
        <input type="number" name="total_copies" id="total_copies"
            min="1" required
            value="{{ old('total_copies', 1) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
        focus:border-indigo-500 focus:ring-indigo-500">
    </div>
</div>

{{-- Fila 6: categoría --}}
<div>
    <label for="category_id" class="block text-sm font-medium text-slate-
700">
        Categoría <span class="text-red-600">*</span>
    </label>
    <select name="category_id" id="category_id" required
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">
        <option value="">-- Seleccionar categoría --</option>
        @foreach ($categories as $category)
            <option value="{{ $category->id }}"
                {{ old('category_id') == $category->id ? 'selected' : '' }}>
                {{ $category->name }}
            </option>
        @endforeach
    </select>
</div>

{{-- Fila 7: autores (multi-select con checkboxes) --}}
<div>
    <label class="block text-sm font-medium text-slate-700 mb-2">
        Autores <span class="text-red-600">*</span>
    </label>
    <div class="grid grid-cols-1 md:grid-cols-2 gap-2 max-h-64 overflow-y-
auto
        border border-slate-200 rounded-md p-3 bg-slate-50">
        @foreach ($authors as $author)
            <label class="flex items-center gap-2 text-sm text-slate-700">
                <input type="checkbox" name="authors[]" value="{{ $author-
>id }}"
                {{ in_array($author->id, old('authors', [])) ? 'checked'
: '' }}
                class="rounded border-slate-300 text-indigo-600
                focus:ring-indigo-500">
                {{ $author->first_name }} {{ $author->last_name }}
            </label>
        @endforeach
    </div>
    <p class="mt-1 text-xs text-slate-500">
        Marca uno o más autores. El nombre se enviará como arreglo
        authors[].
    </p>
</div>

```

```

    {{-- Botones --}}
    <div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-
200">
        <a href="{{ route('books.index') }}"
            class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
            border border-slate-300 rounded-md hover:bg-slate-50">
            Cancelar
        </a>
        <button type="submit"
            class="px-4 py-2 text-sm font-medium text-white
            bg-indigo-600 border border-transparent rounded-md
            hover:bg-indigo-700">
            Guardar Libro
        </button>
    </div>
</form>
</div>
@endsection

```

DEFINICIÓN Mecanismo de protección CSRF en Laravel

CSRF (Cross-Site Request Forgery, falsificación de petición entre sitios) es un ataque en el cual un sitio malicioso induce al navegador del usuario a enviar una petición autenticada a otro sitio en el que ya tiene sesión iniciada. Si la víctima es administradora de la biblioteca y visita una página maliciosa que contiene un formulario oculto que envía un POST a `/books/{id}/destroy`, el navegador adjuntará automáticamente las cookies de sesión y Laravel tendría que decidir si confiar en esa petición.

Para evitarlo, Laravel emplea el patrón synchronizer token: en cada sesión genera un token aleatorio y exige que toda petición que modifique el estado (POST, PUT, PATCH, DELETE) lo presente. Como el sitio malicioso no puede leer ese token (la política same-origin del navegador se lo impide), el ataque queda neutralizado.

La directiva `@csrf` imprime un campo oculto `<input type="hidden" name="_token" value="...">` con el token de la sesión actual. El middleware `PreventRequestForgery` (en Laravel 13 reemplaza al antiguo `VerifyCsrfToken`) compara ese campo con el token guardado en sesión y rechaza la petición con un error 419 si no coinciden.

3.1.2 Ajustar el método `create()` del controlador

El formulario espera dos colecciones: `$categories` y `$authors`. Asegúrate de que `BookController@create` las pase a la vista:

```

// php
// app/Http/Controllers/BookController.php

```



```
use App\Models\Author;
use App\Models\Category;

public function create()
{
    $categories = Category::orderBy('name')->get();
    $authors    = Author::orderBy('last_name')->get();

    return view('books.create', compact('categories', 'authors'));
}
```

3.2 Método store(): persistir el nuevo libro

Cuando el usuario envía el formulario, Laravel direcciona la petición POST /books al método store() del controlador. Este método debe: validar los datos, calcular las copias disponibles, crear el libro y sincronizar los autores asociados.

```
// php
// app/Http/Controllers/BookController.php

use Illuminate\Http\Request;
use App\Models\Book;

public function store(Request $request)
{
    // 1. Validación inline (la mejoraremos en la Guía 5 con Form Requests)
    $validated = $request->validate([
        'title'      => 'required|string|max:255',
        'isbn'       => 'required|string|size:13|unique:books,isbn',
        'publisher'  => 'nullable|string|max:200',
        'publish_year' => 'nullable|integer|min:1000|max:' . date('Y'),
        'pages'      => 'nullable|integer|min:1',
        'language'   => 'nullable|string|max:30',
        'description' => 'nullable|string',
        'cover_url'  => 'nullable|url|max:500',
        'total_copies' => 'required|integer|min:1',
        'category_id' => 'required|exists:categories,id',
        'authors'    => 'required|array|min:1',
        'authors.*'  => 'integer|exists:authors,id',
    ]);

    // 2. Al crear un libro nuevo, todas las copias están disponibles
    $validated['available_copies'] = $validated['total_copies'];

    // 3. Crear el libro
    $book = Book::create($validated);

    // 4. Asociar autores (relación N:M) usando sync()
    $book->authors()->sync($request->input('authors', []));

    // 5. Mensaje flash y redirección
    session()->flash('success', 'Libro registrado exitosamente.');
```

```
return redirect()->route('books.show', $book);
}
```

NOTA ¿Por qué sync() y no attach()?

`attach()` agrega registros a la tabla pivote sin verificar duplicados, mientras que `sync()` reemplaza por completo el conjunto de relaciones. Para un libro recién creado el resultado es idéntico, pero usar `sync()` desde el inicio nos permite reutilizar la misma línea en el método `update()`, donde sí necesitamos el comportamiento de reemplazo total.

ADVERTENCIA Asignación masiva: \$fillable

Para que `Book::create($validated)` funcione, todos los campos deben estar declarados en la propiedad `$fillable` del modelo. Si recibes el error `MassAssignmentException`, revisa que `app/Models/Book.php` incluya todos los nombres usados en el formulario:

```
protected $fillable =  
['title','isbn','publisher','publish_year','pages','language','description','cover_url','total_copies','available_copies','category_id'];
```

3.3 Formulario de edición de libros

La vista de edición es muy similar a la de creación, con tres diferencias clave: el formulario debe pre-llenarse con los valores actuales del libro, debe enviar el método HTTP PUT (no POST), y el campo isbn se muestra en modo lectura para no romper referencias bibliográficas.

3.3.1 Método edit() del controlador

```
// php
// app/Http/Controllers/BookController.php

public function edit(Book $book)
{
    $categories = Category::orderBy('name')->get();
    $authors    = Author::orderBy('last_name')->get();

    return view('books.edit', compact('book', 'categories', 'authors'));
}
```

3.3.2 Vista resources/views/books/edit.blade.php

Crea el archivo books/edit.blade.php con el siguiente contenido. Observa cómo cada input usa `old('campo', $book->campo)`: `old()` recupera el valor enviado tras un fallo de validación, y como segundo argumento ofrece el valor actual del libro como respaldo.

```
// blade
{{-- resources/views/books/edit.blade.php --}}
@extends('layouts.app')

@section('title', 'Editar libro')

@section('content')


<div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Editar libro</h1>
        <a href="{{ route('books.show', $book) }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al libro</a>
    </div>

    @if ($errors->any())
        <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
            <p class="font-semibold text-red-800 mb-2">
                Hay {{ $errors->count() }} error(es) en el formulario:
            </p>
            <ul class="list-disc list-inside text-sm text-red-700">
                @foreach ($errors->all() as $error)
                    <li>{{ $error }}</li>
                @endforeach
            </ul>
        </div>
    @endif


```

```

{{-- IDs de autores actualmente asociados al libro --}}
@php
    $selectedAuthorIds = $book->authors->pluck('id')->toArray();
@endphp

<form method="POST" action="{{ route('books.update', $book) }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    @csrf
    @method('PUT')

    {{-- Título --}}
    <div>
        <label for="title" class="block text-sm font-medium text-slate-700">
            Título <span class="text-red-600">*</span>
        </label>
        <input type="text" name="title" id="title" required
            value="{{ old('title', $book->title) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                focus:border-indigo-500 focus:ring-indigo-500">
    </div>

    {{-- ISBN (solo lectura) y editorial --}}
    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="isbn" class="block text-sm font-medium text-slate-700">
                ISBN
            </label>
            <input type="text" id="isbn" readonly
                value="{{ $book->isbn }}"
                class="mt-1 block w-full rounded-md border-slate-300
                    bg-slate-100 text-slate-500 cursor-not-allowed">
            <p class="mt-1 text-xs text-amber-700">
                El ISBN no se puede modificar para preservar las referencias
                bibliográficas.
            </p>
        </div>
        <div>
            <label for="publisher" class="block text-sm font-medium text-slate-
700">
                Editorial
            </label>
            <input type="text" name="publisher" id="publisher"
                value="{{ old('publisher', $book->publisher) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
    </div>

    {{-- Año, páginas, idioma --}}
    <div class="grid grid-cols-1 md:grid-cols-3 gap-6">
        <div>
            <label for="publish_year" class="block text-sm font-medium text-
slate-700">
                Año de publicación
            </label>
            <input type="number" name="publish_year" id="publish_year"
                min="1000" max="{{ date('Y') }}"

```

```

        value="{{ old('publish_year', $book->publish_year) }}"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                                focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="pages" class="block text-sm font-medium text-slate-700">
            Páginas
        </label>
        <input type="number" name="pages" id="pages" min="1"
            value="{{ old('pages', $book->pages) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                                focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="language" class="block text-sm font-medium text-slate-700">
            Idioma
        </label>
        <select name="language" id="language"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                                focus:border-indigo-500 focus:ring-indigo-500">
            @php
                $idiomas = ['Español', 'Inglés', 'Portugués', 'Francés',
'Alemán', 'Otro'];
            @endphp
            @foreach ($idiomas as $idioma)
                <option value="{{ $idioma }}"
                    {{ old('language', $book->language) === $idioma ?
'selected' : '' }}>
                    {{ $idioma }}
                </option>
            @endforeach
        </select>
    </div>
</div>

{{-- Descripción --}}
<div>
    <label for="description" class="block text-sm font-medium text-slate-700">
        Descripción
    </label>
    <textarea name="description" id="description" rows="4"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                focus:border-indigo-500 focus:ring-indigo-500">{{
old('description', $book->description) }}</textarea>
    </div>

{{-- Portada y copias totales --}}
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
    <div>
        <label for="cover_url" class="block text-sm font-medium text-slate-700">
            URL de la portada
        </label>
        <input type="url" name="cover_url" id="cover_url"
            value="{{ old('cover_url', $book->cover_url) }}"

```

```

        class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
        focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="total_copies" class="block text-sm font-medium text-
slate-700">
            Copias totales <span class="text-red-600">*</span>
        </label>
        <input type="number" name="total_copies" id="total_copies"
            min="1" required
            value="{{ old('total_copies', $book->total_copies) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
            focus:border-indigo-500 focus:ring-indigo-500">
        <p class="mt-1 text-xs text-slate-500">
            Disponibles actualmente: {{ $book->available_copies }} de {{
$book->total_copies }}.
        </p>
    </div>
</div>

{{-- Categoría --}}
<div>
    <label for="category_id" class="block text-sm font-medium text-slate-
700">
        Categoría <span class="text-red-600">*</span>
    </label>
    <select name="category_id" id="category_id" required
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">
        <option value="">-- Seleccionar categoría --</option>
        @foreach ($categories as $category)
            <option value="{{ $category->id }}"
                {{ old('category_id', $book->category_id) == $category->id ?
'selected' : '' }}>
                {{ $category->name }}
            </option>
        @endforeach
    </select>
</div>

{{-- Autores: pre-marcados los que ya están asociados --}}
<div>
    <label class="block text-sm font-medium text-slate-700 mb-2">
        Autores <span class="text-red-600">*</span>
    </label>
    <div class="grid grid-cols-1 md:grid-cols-2 gap-2 max-h-64 overflow-y-
auto
        border border-slate-200 rounded-md p-3 bg-slate-50">
        @foreach ($authors as $author)
            <label class="flex items-center gap-2 text-sm text-slate-700">
                <input type="checkbox" name="authors[]" value="{{ $author-
>id }}"
                    {{ in_array($author->id, old('authors',
$selectdAuthorIds)) ? 'checked' : '' }}
                class="rounded border-slate-300 text-indigo-600
                focus:ring-indigo-500">
                {{ $author->first_name }} {{ $author->last_name }}
            </label>

```

```

        @endforeach
      </div>
    </div>

    <div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-
200">
      <a href="{{ route('books.show', $book) }}"
        class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
          border border-slate-300 rounded-md hover:bg-slate-50">
        Cancelar
      </a>
      <button type="submit"
        class="px-4 py-2 text-sm font-medium text-white
          bg-indigo-600 border border-transparent rounded-md
          hover:bg-indigo-700">
        Actualizar Libro
      </button>
    </div>
  </form>
</div>
@endsection

```

NOTA El truco de @method('PUT')

Los formularios HTML solo soportan los métodos GET y POST. Para enviar PUT, PATCH o DELETE, Laravel utiliza `@method('PUT')` que imprime un campo oculto `<input type="hidden" name="_method" value="PUT">`. El middleware `MethodOverride` lee ese campo y enruta la petición como si fuera un PUT real, lo que permite usar `Route::put()` y `Route::resource()` con formularios HTML estándar.

3.4 Método update(): persistir cambios

El método update() valida los datos, recalcula las copias disponibles si cambió el total de copias, actualiza el modelo y sincroniza los autores.

```
// php
// app/Http/Controllers/BookController.php

public function update(Request $request, Book $book)
{
    // 1. Validación: en isbn ignoramos el registro actual,
    // pero igualmente lo dejamos fuera del array $validated más abajo.
    $validated = $request->validate([
        'title'      => 'required|string|max:255',
        'publisher'  => 'nullable|string|max:200',
        'publish_year' => 'nullable|integer|min:1000|max:' . date('Y'),
        'pages'      => 'nullable|integer|min:1',
        'language'   => 'nullable|string|max:30',
        'description' => 'nullable|string',
        'cover_url'   => 'nullable|url|max:500',
        'total_copies' => 'required|integer|min:1',
        'category_id' => 'required|exists:categories,id',
        'authors'     => 'required|array|min:1',
        'authors.*'   => 'integer|exists:authors,id',
    ]);

    // 2. Recalcular available_copies de forma proporcional si cambió total_copies.
    // Ejemplo: tenía 5 totales y 2 prestadas (3 disponibles). Si pasa a 8
    // totales, las prestadas siguen siendo 2 -> available_copies = 8 - 2 = 6.
    $loanedCopies = $book->total_copies - $book->available_copies;
    $newTotal     = $validated['total_copies'];
    $validated['available_copies'] = max(0, $newTotal - $loanedCopies);

    // 3. Actualizar el libro (no incluimos isbn porque no es editable)
    $book->update($validated);

    // 4. Sincronizar autores: sync() reemplaza el conjunto completo
    $book->authors()->sync($request->input('authors', []));

    // 5. Mensaje flash y redirección
    session()->flash('success', 'Libro actualizado correctamente.');
```

```
return redirect()->route('books.show', $book);
}
```

EJEMPLO Validación unique con excepción

Aunque en este formulario el ISBN no se edita, en otros casos necesitarás validar que un campo siga siendo único pero ignorando el propio registro. La regla se escribe así:

```
'isbn' => 'required|string|size:13|unique:books,isbn,' . $book->id
```

Sin esa exclusión, Laravel reportaría que el ISBN ya existe... porque coincidiría con el del propio libro que estamos editando.

3.5 Eliminar un libro (soft delete con verificación)

La eliminación física de registros bibliográficos suele ser una mala idea: los préstamos históricos pierden su referencia y los reportes quedan inconsistentes. Por eso aplicamos soft delete (eliminación lógica). Además, antes de eliminar verificamos que el libro no tenga préstamos activos.

3.5.1 Asegurar que el modelo Book usa SoftDeletes

Si aún no lo hiciste en la Guía 6, agrega el trait SoftDeletes al modelo:

```
// php
// app/Models/Book.php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Database\Eloquent\Factories\HasFactory;

class Book extends Model
{
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'title', 'isbn', 'publisher', 'publish_year', 'pages', 'language',
        'description', 'cover_url', 'total_copies', 'available_copies', 'category_id',
    ];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function authors()
    {
        return $this->belongsToMany(Author::class)->withTimestamps();
    }

    public function loans()
    {
        return $this->hasMany(Loan::class);
    }

    // Préstamos no devueltos
    public function activeLoans()
    {
        return $this->hasMany(Loan::class)->whereNull('returned_at');
    }
}
```

La columna `deleted_at` debe existir en la tabla `books` (la creaste en una migración previa con `$table->softDeletes()`). Si no la tienes, genera y ejecuta una migración:

```
// bash
php artisan make:migration add_softdeletes_to_books_table --table=books

# En la migración:
public function up(): void
{
    Schema::table('books', function (Blueprint $table) {
        $table->softDeletes();
    });
}

php artisan migrate
```

3.5.2 Método destroy() del controlador

```
// php
// app/Http/Controllers/BookController.php

public function destroy(Book $book)
{
    // 1. Verificar que no tenga préstamos activos
    if ($book->activeLoans()->count() > 0) {
        return back()->with(
            'error',
            'No se puede eliminar un libro con préstamos activos.'
        );
    }

    // 2. Soft delete: marca deleted_at, no borra físicamente
    $book->delete();

    session()->flash('success', 'Libro eliminado correctamente.');
```

return redirect()->route('books.index');

```
}
```

3.5.3 Botón de eliminar en books/show.blade.php

En la vista de detalle agregamos un formulario con método DELETE y una confirmación nativa del navegador. El botón se estiliza en rojo para señalar acción destructiva.

```
// blade
{{-- resources/views/books/show.blade.php - fragmento al final --}}
<div class="mt-6 flex items-center gap-3">
    <a href="{{ route('books.edit', $book) }}"
        class="px-4 py-2 text-sm font-medium text-white bg-indigo-600
            rounded-md hover:bg-indigo-700">
        Editar
    </a>

    <form method="POST" action="{{ route('books.destroy', $book) }}"
        onsubmit="return confirm('¿Está seguro de eliminar este libro?');">
        @csrf
```

```
@method('DELETE')
<button type="submit"
        class="px-4 py-2 text-sm font-medium text-white bg-red-600
                rounded-md hover:bg-red-700">
    Eliminar
</button>
</form>
</div>
```

ADVERTENCIA El return back() y los mensajes flash

`back()` devuelve al usuario a la página anterior (en este caso la vista de detalle del libro). El método `->with('error', '...')` adjunta un mensaje flash que solo persiste para la siguiente petición. En la sección 3.7 mostraremos cómo el layout principal renderiza estos mensajes con estilos diferenciados según el tipo (success en verde, error en rojo).

3.6 CRUD completo de autores

Replicamos la misma estructura para autores, con campos más sencillos: nombre, apellido, nacionalidad, fecha de nacimiento y biografía. La eliminación verifica que el autor no tenga libros asociados.

3.6.1 AuthorController completo

```
// php
<?php
// app/Http/Controllers/AuthorController.php

namespace App\Http\Controllers;

use App\Models\Author;
use Illuminate\Http\Request;

class AuthorController extends Controller
{
    public function index()
    {
        $authors = Author::orderBy('last_name')->paginate(15);
        return view('authors.index', compact('authors'));
    }

    public function show(Author $author)
    {
        $author->load('books');
        return view('authors.show', compact('author'));
    }

    public function create()
    {
        return view('authors.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'first_name' => 'required|string|max:100',
            'last_name'  => 'required|string|max:100',
            'nationality' => 'nullable|string|max:80',
            'birth_date' => 'nullable|date|before:today',
            'biography'  => 'nullable|string',
        ]);

        $author = Author::create($validated);

        session()->flash('success', 'Autor registrado correctamente. ');
        return redirect()->route('authors.show', $author);
    }

    public function edit(Author $author)
    {
        return view('authors.edit', compact('author'));
    }
}
```

```
public function update(Request $request, Author $author)
{
    $validated = $request->validate([
        'first_name' => 'required|string|max:100',
        'last_name'  => 'required|string|max:100',
        'nationality' => 'nullable|string|max:80',
        'birth_date' => 'nullable|date|before:today',
        'biography'  => 'nullable|string',
    ]);

    $author->update($validated);

    session()->flash('success', 'Autor actualizado correctamente.');
```

```
return redirect()->route('authors.show', $author);
}

public function destroy(Author $author)
{
    // Verificar integridad referencial: no eliminar si tiene libros
    if ($author->books()->count() > 0) {
        return back()->with(
            'error',
            'No se puede eliminar un autor con libros asociados.'
        );
    }

    $author->delete();

    session()->flash('success', 'Autor eliminado correctamente.');
```

```
return redirect()->route('authors.index');
}
}
```

3.6.2 Vista resources/views/authors/create.blade.php

```
// blade
{{-- resources/views/authors/create.blade.php --}}
@extends('layouts.app')

@section('title', 'Registrar autor')

@section('content')


<div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Registrar autor</h1>
        <a href="{{ route('authors.index') }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al
listado</a>
    </div>

    @if ($errors->any())
        <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
            <ul class="list-disc list-inside text-sm text-red-700">


```

```

        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>
</div>
@endif

<form method="POST" action="{{ route('authors.store') }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    @csrf

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="first_name" class="block text-sm font-medium text-slate-
700">
                Nombre <span class="text-red-600">*</span>
            </label>
            <input type="text" name="first_name" id="first_name" required
                value="{{ old('first_name') }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                    focus:border-indigo-500 focus:ring-indigo-500">
            </div>
            <div>
                <label for="last_name" class="block text-sm font-medium text-slate-
700">
                    Apellido <span class="text-red-600">*</span>
                </label>
                <input type="text" name="last_name" id="last_name" required
                    value="{{ old('last_name') }}"
                    class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                        focus:border-indigo-500 focus:ring-indigo-500">
                </div>
            </div>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="nationality" class="block text-sm font-medium text-
slate-700">
                Nacionalidad
            </label>
            <input type="text" name="nationality" id="nationality"
                value="{{ old('nationality') }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                    focus:border-indigo-500 focus:ring-indigo-500">
            </div>
            <div>
                <label for="birth_date" class="block text-sm font-medium text-slate-
700">
                    Fecha de nacimiento
                </label>
                <input type="date" name="birth_date" id="birth_date"
                    max="{{ date('Y-m-d') }}"
                    value="{{ old('birth_date') }}"
                    class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
                        focus:border-indigo-500 focus:ring-indigo-500">
                </div>
            </div>
    </div>

```



```

    </div>

    <div>
        <label for="biography" class="block text-sm font-medium text-slate-700">
            Biografía
        </label>
        <textarea name="biography" id="biography" rows="5"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                focus:border-indigo-500 focus:ring-indigo-500">{{
old('biography') }}</textarea>
    </div>

    <div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-
200">
        <a href="{{ route('authors.index') }}"
            class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
                border border-slate-300 rounded-md hover:bg-slate-50">
            Cancelar
        </a>
        <button type="submit"
            class="px-4 py-2 text-sm font-medium text-white
                bg-indigo-600 border border-transparent rounded-md
                hover:bg-indigo-700">
            Guardar Autor
        </button>
    </div>
</form>
</div>
@endsection

```

3.6.3 Vista resources/views/authors/edit.blade.php

Es prácticamente idéntica a create, pero con `@method('PUT')`, pre-llenado con `old('campo', $author->campo)` y action apuntando a `authors.update`. Más adelante la refactorizaremos en un partial compartido.

```

// blade
{{-- resources/views/authors/edit.blade.php --}}
@extends('layouts.app')

@section('title', 'Editar autor')

@section('content')
<div class="max-w-3xl mx-auto px-4 py-8">

    <div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Editar autor</h1>
        <a href="{{ route('authors.show', $author) }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al autor</a>
    </div>

    @if ($errors->any())
        <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
            <ul class="list-disc list-inside text-sm text-red-700">

```

```

        @foreach ($errors->all() as $error)
            <li>{{ $error }}</li>
        @endforeach
    </ul>
</div>
@endif

<form method="POST" action="{{ route('authors.update', $author) }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    @csrf
    @method('PUT')

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="first_name" class="block text-sm font-medium text-slate-700">
                Nombre <span class="text-red-600">*</span>
            </label>
            <input type="text" name="first_name" id="first_name" required
                value="{{ old('first_name', $author->first_name) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
            </div>
            <div>
            <label for="last_name" class="block text-sm font-medium text-slate-700">
                Apellido <span class="text-red-600">*</span>
            </label>
            <input type="text" name="last_name" id="last_name" required
                value="{{ old('last_name', $author->last_name) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
            </div>
        </div>

        <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div>
            <label for="nationality" class="block text-sm font-medium text-slate-700">
                Nacionalidad
            </label>
            <input type="text" name="nationality" id="nationality"
                value="{{ old('nationality', $author->nationality) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
            </div>
            <div>
            <label for="birth_date" class="block text-sm font-medium text-slate-700">
                Fecha de nacimiento
            </label>
            <input type="date" name="birth_date" id="birth_date"
                max="{{ date('Y-m-d') }}"
                value="{{ old('birth_date', optional($author->birth_date)->format('Y-m-d')) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm

```

```

                focus:border-indigo-500 focus:ring-indigo-500">
            </div>
        </div>

        <div>
            <label for="biography" class="block text-sm font-medium text-slate-700">
                Biografía
            </label>
            <textarea name="biography" id="biography" rows="5"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">{{
old('biography', $author->biography) }}</textarea>
        </div>

        <div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-
200">
            <a href="{{ route('authors.show', $author) }}"
                class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
                    border border-slate-300 rounded-md hover:bg-slate-50">
                Cancelar
            </a>
            <button type="submit"
                class="px-4 py-2 text-sm font-medium text-white
                    bg-indigo-600 border border-transparent rounded-md
                    hover:bg-indigo-700">
                Actualizar Autor
            </button>
        </div>
    </form>
</div>
@endsection

```

3.7 Refactorización con partials `_form.blade.php`

Las vistas create y edit comparten más del 90% del código. Esta duplicación es una bomba de tiempo: cada vez que agreguemos un campo, tendríamos que modificarlo en dos lugares y es muy fácil olvidar uno. La solución es extraer el formulario a un partial reutilizable.

DEFINICIÓN Partials en Blade

Un partial es una vista parcial que se incluye dentro de otra mediante `@include('ruta.del.partial', ['var' => $valor])`. Por convención, los partials se nombran con guion bajo inicial (`_form.blade.php`) para distinguirlos visualmente de las vistas principales. Las variables del scope padre se heredan automáticamente, pero es buena práctica pasarlas explícitamente para que el partial sea autocontenido y reutilizable.

3.7.1 Partial `resources/views/books/_form.blade.php`

Creamos el partial usando una variable `$book` que será null en el caso de creación y el modelo en edición. La lógica condicional define la acción, el método y el texto del botón.

```
// blade
{{-- resources/views/books/_form.blade.php --}}
{{--
    Variables esperadas:
    - $book          Book|null      (null en create, modelo en edit)
    - $categories    Collection     (todas las categorías)
    - $authors       Collection     (todos los autores)
--}}
@php
    $isEdit = isset($book) && $book && $book->exists;
    $action = $isEdit ? route('books.update', $book) : route('books.store');
    $submit = $isEdit ? 'Actualizar Libro' : 'Guardar Libro';
    $selectedAuthorIds = $isEdit ? $book->authors->pluck('id')->toArray() : [];
@endphp

@if ($errors->any())
    <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
        <p class="font-semibold text-red-800 mb-2">
            Hay {{ $errors->count() }} error(es) en el formulario:
        </p>
        <ul class="list-disc list-inside text-sm text-red-700">
            @foreach ($errors->all() as $error)
                <li>{{ $error }}</li>
            @endforeach
        </ul>
    </div>
@endif

<form method="POST" action="{{ $action }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    @csrf
    @if ($isEdit)
        @method('PUT')
```

```

@endif

{{-- Título --}}
<div>
  <label for="title" class="block text-sm font-medium text-slate-700">
    Título <span class="text-red-600">*</span>
  </label>
  <input type="text" name="title" id="title" required
    value="{{ old('title', $book->title) }}"
    class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
      focus:border-indigo-500 focus:ring-indigo-500">
</div>

{{-- ISBN --}}
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
  <div>
    <label for="isbn" class="block text-sm font-medium text-slate-700">
      ISBN @if (!$isEdit)<span class="text-red-600">*</span>@endif
    </label>
    @if ($isEdit)
      <input type="text" id="isbn" readonly value="{{ $book->isbn }}"
        class="mt-1 block w-full rounded-md border-slate-300
          bg-slate-100 text-slate-500 cursor-not-allowed">
      <p class="mt-1 text-xs text-amber-700">El ISBN no es editable.</p>
    @else
      <input type="text" name="isbn" id="isbn" required
        placeholder="978-..." value="{{ old('isbn') }}"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-
sm
          focus:border-indigo-500 focus:ring-indigo-500">
    @endif
  </div>
  <div>
    <label for="publisher" class="block text-sm font-medium text-slate-700">
      Editorial
    </label>
    <input type="text" name="publisher" id="publisher"
      value="{{ old('publisher', $book->publisher) }}"
      class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">
  </div>
</div>

{{-- Año, páginas, idioma --}}
<div class="grid grid-cols-1 md:grid-cols-3 gap-6">
  <div>
    <label for="publish_year" class="block text-sm font-medium text-slate-
700">
      Año
    </label>
    <input type="number" name="publish_year" id="publish_year"
      min="1000" max="{{ date('Y') }}"
      value="{{ old('publish_year', $book->publish_year) }}"
      class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">
  </div>
  <div>
    <label for="pages" class="block text-sm font-medium text-slate-
700">Páginas</label>
    <input type="number" name="pages" id="pages" min="1"

```

```

        value="{{ old('pages', $book?->pages) }}"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
              focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="language" class="block text-sm font-medium text-slate-
700">Idioma</label>
        <select name="language" id="language"
              class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
              focus:border-indigo-500 focus:ring-indigo-500">
            @php $idiomas =
['Español','Inglés','Portugués','Francés','Alemán','Otro']; @endphp
            @foreach ($idiomas as $idioma)
                <option value="{{ $idioma }}"
                    {{ old('language', $book?->language) === $idioma ?
'selected' : '' }}>
                    {{ $idioma }}
                </option>
            @endforeach
        </select>
    </div>
</div>

{{-- Descripción --}}
<div>
    <label for="description" class="block text-sm font-medium text-slate-
700">Descripción</label>
    <textarea name="description" id="description" rows="4"
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
        focus:border-indigo-500 focus:ring-indigo-500">{{
old('description', $book?->description) }}</textarea>
</div>

{{-- Portada y copias totales --}}
<div class="grid grid-cols-1 md:grid-cols-2 gap-6">
    <div>
        <label for="cover_url" class="block text-sm font-medium text-slate-
700">URL de portada</label>
        <input type="url" name="cover_url" id="cover_url"
            value="{{ old('cover_url', $book?->cover_url) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
            focus:border-indigo-500 focus:ring-indigo-500">
    </div>
    <div>
        <label for="total_copies" class="block text-sm font-medium text-slate-
700">
            Copias totales <span class="text-red-600">*</span>
        </label>
        <input type="number" name="total_copies" id="total_copies"
            min="1" required
            value="{{ old('total_copies', $book?->total_copies ?? 1) }}"
            class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
            focus:border-indigo-500 focus:ring-indigo-500">
    </div>
</div>

{{-- Categoría --}}
<div>
    <label for="category_id" class="block text-sm font-medium text-slate-700">
        Categoría <span class="text-red-600">*</span>

```

```

</label>
<select name="category_id" id="category_id" required
        class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
              focus:border-indigo-500 focus:ring-indigo-500">
  <option value="">-- Seleccionar categoría --</option>
  @foreach ($categories as $category)
    <option value="{{ $category->id }}"
      {{ old('category_id', $book->category_id) == $category->id ?
'selected' : '' }}>
      {{ $category->name }}
    </option>
  @endforeach
</select>
</div>

{{-- Autores --}}
<div>
  <label class="block text-sm font-medium text-slate-700 mb-2">
    Autores <span class="text-red-600">*</span>
  </label>
  <div class="grid grid-cols-1 md:grid-cols-2 gap-2 max-h-64 overflow-y-auto
    border border-slate-200 rounded-md p-3 bg-slate-50">
    @foreach ($authors as $author)
      <label class="flex items-center gap-2 text-sm text-slate-700">
        <input type="checkbox" name="authors[]" value="{{ $author->id
}}
        {{ in_array($author->id, old('authors', $selectedAuthorIds))
? 'checked' : '' }}
        class="rounded border-slate-300 text-indigo-600
          focus:ring-indigo-500">
        {{ $author->first_name }} {{ $author->last_name }}
      </label>
    @endforeach
  </div>
</div>

{{-- Botones --}}
<div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-200">
  <a href="{{ $isEdit ? route('books.show', $book) : route('books.index') }}"
    class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
      border border-slate-300 rounded-md hover:bg-slate-50">
    Cancelar
  </a>
  <button type="submit"
    class="px-4 py-2 text-sm font-medium text-white
      bg-indigo-600 border border-transparent rounded-md
      hover:bg-indigo-700">
    {{ $submit }}
  </button>
</div>
</form>

```

3.7.2 create.blade.php y edit.blade.php quedan minimalistas

```

// blade
{{-- resources/views/books/create.blade.php (versión refactorizada) --}}
@extends('layouts.app')

```

```

@section('title', 'Registrar libro')

@section('content')
<div class="max-w-4xl mx-auto px-4 py-8">
    <div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Registrar libro</h1>
        <a href="{{ route('books.index') }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al
catálogo</a>
    </div>

    @include('books._form', [
        'book' => null,
        'categories' => $categories,
        'authors' => $authors,
    ])
</div>
@endsection

```

```

// blade
{{-- resources/views/books/edit.blade.php (versión refactorizada) --}}
@extends('layouts.app')

@section('title', 'Editar libro')

@section('content')
<div class="max-w-4xl mx-auto px-4 py-8">
    <div class="mb-6 flex items-center justify-between">
        <h1 class="text-3xl font-bold text-slate-800">Editar libro</h1>
        <a href="{{ route('books.show', $book) }}"
            class="text-sm text-slate-600 hover:text-slate-900">← Volver al libro</a>
    </div>

    @include('books._form', [
        'book' => $book,
        'categories' => $categories,
        'authors' => $authors,
    ])
</div>
@endsection

```

3.7.3 Partial resources/views/authors/_form.blade.php

Aplicamos el mismo patrón al formulario de autores.

```

// blade
{{-- resources/views/authors/_form.blade.php --}}
@php
    $isEdit = isset($author) && $author && $author->exists;
    $action = $isEdit ? route('authors.update', $author) : route('authors.store');
    $submit = $isEdit ? 'Actualizar Autor' : 'Guardar Autor';
    $cancelUrl = $isEdit ? route('authors.show', $author) : route('authors.index');

```



```

@endphp

@if ($errors->any())
    <div class="mb-6 rounded-md border border-red-300 bg-red-50 p-4">
        <ul class="list-disc list-inside text-sm text-red-700">
            @foreach ($errors->all() as $error)<li>{{ $error }}</li>@endforeach
        </ul>
    </div>
@endif

<form method="POST" action="{{ $action }}"
    class="bg-white shadow rounded-lg p-6 space-y-6">
    @csrf
    @if ($isEdit) @method('PUT') @endif

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="first_name" class="block text-sm font-medium text-slate-700">
                Nombre <span class="text-red-600">*</span></label>
            <input type="text" name="first_name" id="first_name" required
                value="{{ old('first_name', $author?->first_name) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
        <div>
            <label for="last_name" class="block text-sm font-medium text-slate-700">
                Apellido <span class="text-red-600">*</span></label>
            <input type="text" name="last_name" id="last_name" required
                value="{{ old('last_name', $author?->last_name) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
    </div>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
        <div>
            <label for="nationality" class="block text-sm font-medium text-slate-700">Nacionalidad</label>
            <input type="text" name="nationality" id="nationality"
                value="{{ old('nationality', $author?->nationality) }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
        <div>
            <label for="birth_date" class="block text-sm font-medium text-slate-700">Fecha de nacimiento</label>
            <input type="date" name="birth_date" id="birth_date" max="{{ date('Y-m-d') }}"
                value="{{ old('birth_date', $isEdit ? optional($author->birth_date)->format('Y-m-d') : '') }}"
                class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
                    focus:border-indigo-500 focus:ring-indigo-500">
        </div>
    </div>

    <div>

```

```
<label for="biography" class="block text-sm font-medium text-slate-700">Biografía</label>
<textarea name="biography" id="biography" rows="5"
  class="mt-1 block w-full rounded-md border-slate-300 shadow-sm
    focus:border-indigo-500 focus:ring-indigo-500">{{
old('biography', $author?->biography) }}</textarea>
</div>

<div class="flex items-center justify-end gap-3 pt-4 border-t border-slate-200">
  <a href="{{ $cancelUrl }}"
    class="px-4 py-2 text-sm font-medium text-slate-700 bg-white
      border border-slate-300 rounded-md hover:bg-slate-50">
    Cancelar
  </a>
  <button type="submit"
    class="px-4 py-2 text-sm font-medium text-white
      bg-indigo-600 border border-transparent rounded-md
      hover:bg-indigo-700">
    {{ $submit }}
  </button>
</div>
</form>
```

3.8 Mensajes flash mejorados en el layout

Centralizamos en el layout principal la presentación de los mensajes flash, de modo que todos los controladores se beneficien del mismo estilo. Distinguimos visualmente success (verde) de error (rojo) y agregamos auto-dismiss con Alpine.js.

3.8.1 Asegurar que Alpine.js esté disponible

Si tu proyecto fue iniciado con Breeze, Alpine ya está incluido. En caso contrario, agrégalo en `resources/js/app.js`:

```
// javascript
// resources/js/app.js
import './bootstrap';

import Alpine from 'alpinejs';

window.Alpine = Alpine;
Alpine.start();
```

Y luego instala la dependencia y reconstruye:

```
// bash
npm install alpinejs
npm run dev
```

3.8.2 Partial `_alerts.blade.php`

Para mantener limpio el layout, extraemos el bloque de alertas a su propio partial:

```
// blade
{{-- resources/views/partials/_alerts.blade.php --}}
@if (session('success'))
    <div x-data="{ open: true }"
        x-show="open"
        x-init="setTimeout(() => open = false, 5000)"
        x-transition.opacity.duration.500ms
        class="mb-4 flex items-start gap-3 rounded-md border border-green-300
            bg-green-50 p-4 text-green-800">
        <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 mt-0.5 shrink-0"
            fill="none" viewBox="0 0 24 24" stroke="currentColor">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                d="M5 13 4 19 11 22 19 19 22 11 19 4 13 5 11 11 22 19 19 22 11 19 4 13 5"/>
            </svg>
        <div class="flex-1 text-sm font-medium">{{ session('success') }}</div>
        <button type="button" @click="open = false"
            class="text-green-600 hover:text-green-900" aria-label="Cerrar">
            ✕
        </div>
@endif
```

```

        </button>
    </div>
@endif

@if (session('error'))
    <div x-data="{ open: true }"
        x-show="open"
        x-init="setTimeout(() => open = false, 7000)"
        x-transition.opacity.duration.500ms
        class="mb-4 flex items-start gap-3 rounded-md border border-red-300
            bg-red-50 p-4 text-red-800">
        <svg xmlns="http://www.w3.org/2000/svg" class="h-5 w-5 mt-0.5 shrink-0"
            fill="none" viewBox="0 0 24 24" stroke="currentColor">
            <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
                d="M6 18L18 6M6 6l12 12"/>
        </svg>
        <div class="flex-1 text-sm font-medium">{{ session('error') }}</div>
        <button type="button" @click="open = false"
            class="text-red-600 hover:text-red-900" aria-label="Cerrar">
            X
        </button>
    </div>
@endif

```

3.8.3 Incluir las alertas en layouts/app.blade.php

Inserta `@include('partials._alerts')` en el layout principal, justo encima de `@yield('content')`:

```

// blade
{{-- resources/views/layouts/app.blade.php - fragmento --}}
<main class="py-8">
    <div class="max-w-7xl mx-auto px-4">
        @include('partials._alerts')
    </div>

    @yield('content')
</main>

```

EJEMPLO Probando los mensajes flash

Después de aplicar estos cambios, navega a `/books/create` y registra un libro. Deberías ver una alerta verde con el mensaje «Libro registrado exitosamente» que se desvanece a los 5 segundos. Si intentas eliminar un libro con préstamos activos, aparecerá una alerta roja con «No se puede eliminar un libro con préstamos activos» que dura 7 segundos.

3.9 Versionado con Git: tag v0.4.0

Concluimos la guía registrando el progreso en Git. Antes del commit, ejecuta una prueba manual rápida del flujo completo: crear un libro, editarlo, intentar eliminarlo (con y sin préstamos) y crear/editar/eliminar un autor.

```
// bash
# 1. Verificar archivos modificados
git status

# 2. Agregar todos los cambios
git add .

# 3. Commit con mensaje descriptivo (Conventional Commits)
git commit -m "feat: CRUD completo para libros y autores con formularios"

# 4. Crear tag anotado para esta entrega
git tag -a v0.4.0 -m "Guía 7: CRUD completo con formularios"

# 5. Empujar commits y tags al remoto
git push origin main --tags
```

NOTA Buenas prácticas de commit

Si realizaste cambios en varias áreas (controladores, vistas, layout), considera fragmentar el commit con `git add -p` para crear commits temáticos más pequeños (`feat: store y update de libros`, `refactor: extraer _form.blade.php`, `feat: alertas con auto-dismiss`). Esto facilita la revisión de código y los rollbacks puntuales.

4. Ejercicios Propuestos

Aplica los conceptos de esta guía resolviendo los siguientes ejercicios. Ambos forman parte de los entregables y se evalúan en la rúbrica.

Ejercicio 1 — CRUD completo de Categorías

Implementa CategoriesController con todos los métodos resource (index, show, create, store, edit, update, destroy) replicando el patrón aplicado a libros y autores. Considera los siguientes requisitos específicos:

1. La ruta debe registrarse como `Route::resource('categories', CategoriesController::class)` en `routes/web.php`.
2. El formulario (create y edit) debe incluir un color picker para el campo color: `<input type="color" name="color">`. Almacena el valor en formato hexadecimal (`#RRGGBB`).
3. La validación debe asegurar que name es único entre categorías (con la excepción correspondiente en update).
4. El método `destroy()` debe verificar que la categoría no tenga libros asociados; en caso contrario, devolver `back()->with('error', '...')`.
5. Refactoriza el formulario en `categories/_form.blade.php` compartido entre create y edit, siguiendo el mismo patrón que en libros y autores.

NOTA Pista para el color picker

El input de tipo `color` envía siempre un valor hexadecimal en minúsculas. Para validarlo, usa `'color' => 'required|regex:/^#[0-9a-fA-F]{6}$/'`. En la vista index, puedes mostrar el color como un círculo decorativo:

```
<span class="inline-block w-4 h-4 rounded-full" style="background-color: {{ $category->color }}"></span>
```

Ejercicio 2 — Restaurar libros eliminados

Aprovecha el soft delete que aplicamos en la sección 3.5 para implementar una vista de papelera con la posibilidad de restaurar libros eliminados:

1. Agrega la ruta POST `/books/{book}/restore` en `routes/web.php` (usa `Route::post(...)->name('books.restore')`).
2. La ruta debe usar Route Model Binding con scope `withTrashed` para encontrar también los libros eliminados: `Route::post('books/{book}/restore', [BookController::class, 'restore']->withTrashed()->name('books.restore'))`;
3. Implementa el método `restore(Book $book)` en `BookController`: `$book->restore()` y redirección a `books.index` con mensaje flash de éxito.
4. Crea la vista `books/trash.blade.php` con la lista de libros eliminados (consulta `Book::onlyTrashed()->get()`) mostrando título, ISBN, fecha de eliminación (`deleted_at`) y un formulario con botón Restaurar.
5. Agrega un enlace «Papelera» en `books/index.blade.php` que apunte a una nueva ruta GET `/books-trash`.

ADVERTENCIA Cuidados con `onlyTrashed()` y `withTrashed()`

`onlyTrashed()` devuelve únicamente los registros con `deleted_at` no nulo. `withTrashed()` incluye eliminados y vigentes. Sin ninguno de los dos, las consultas Eloquent ignoran automáticamente los registros eliminados (gracias al global scope que añade el trait `SoftDeletes`). En las rutas, recuerda encadenar `->withTrashed()` para que el route model binding pueda encontrar los libros eliminados.

5. Rúbrica de Evaluación

Cada criterio se evalúa de forma independiente. La calificación final es la suma de los puntos obtenidos sobre 100.

Criterio	Descripción	Puntos	Obt.
Formulario de creación de libros completo	Vista books/create.blade.php con todos los campos requeridos, validación HTML5, autores como multi-select, categorías dinámicas desde BD y estilos Tailwind.	15	
Método store() con validación y sync de autores	Validación inline con <code>\$request->validate()</code> , cálculo de <code>available_copies</code> , <code>Book::create()</code> y <code>\$book->authors()->sync()</code> . Mensaje flash y redirección a <code>books.show</code> .	15	
Formulario de edición con pre-llenado	Vista books/edit.blade.php con valores actuales mediante <code>old('campo', \$book->campo)</code> , autores pre-marcados con <code>\$book->authors->pluck('id')</code> , <code>@method('PUT')</code> e ISBN readonly.	10	
Método update() con sync y recálculo de copias	Validación, recálculo proporcional de <code>available_copies</code> cuando cambia <code>total_copies</code> , <code>\$book->update()</code> y <code>\$book->authors()->sync()</code> .	10	
Eliminación con verificación de préstamos	Método <code>destroy()</code> que valida <code>\$book->activeLoans()->count()</code> , aplica soft delete y muestra mensaje de error cuando corresponde. Botón en <code>books/show</code> con confirmación.	10	
CRUD completo de autores	AuthorController con store, edit, update y destroy. Vistas create y edit con validación. <code>destroy()</code> bloquea eliminación si el autor tiene libros.	10	
Partials _form.blade.php (books y authors)	Refactorización con partials compartidos entre create y edit. Lógica condicional según <code>\$book/\$author</code> existente o null.	10	
Mensajes flash (success y error)	Partial _alerts.blade.php incluido en el layout, con estilos diferenciados (verde/rojo), íconos SVG y auto-dismiss con Alpine.js.	5	
Ejercicios propuestos	Ejercicio 1 (CRUD de Categorías con color picker) y Ejercicio 2 (papelera y restauración de libros eliminados).	10	
Git tag v0.4.0	Commit semántico, tag anotado v0.4.0 y push a origin main --tags.	5	

	TOTAL	100	—
--	--------------	------------	----------

NOTA Criterios de calidad transversales

Independientemente del puntaje por criterio, se podrá descontar hasta 10 puntos por: código sin formato (uso inconsistente de indentación), nombres de variables o métodos en inglés mal escritos o ambiguos, ausencia de mensajes de commit descriptivos, vistas con HTML mal anidado o inputs sin label asociado por for/id.

6. Entregables y Tag de Git

La entrega de esta guía consiste en lo siguiente:

1. Repositorio del proyecto library-app actualizado en GitHub con el tag v0.4.0 publicado.
2. URL del repositorio enviada a través de la plataforma del curso (un único enlace por equipo o estudiante, según indique el docente).
3. README.md actualizado con una breve descripción de los cambios incorporados en v0.4.0 (CRUD completo, partials, alertas).
4. Capturas de pantalla del flujo: registro de un libro, edición exitosa, intento de eliminación bloqueado por préstamos activos y mensaje flash visible.

Checklist final antes de entregar

- php artisan migrate:fresh --seed se ejecuta sin errores.
- npm run build compila los assets sin advertencias.
- Ningún archivo .env, vendor/ o node_modules/ está versionado.
- php artisan route:list muestra las rutas resource para books, authors y categories.
- git tag --list muestra v0.4.0 y git ls-remote --tags origin lo muestra en GitHub.
- Las vistas se renderizan correctamente en pantallas móviles (probadas con DevTools).

DEFINICIÓN Identificación de la entrega

Tag: `v0.4.0`

Mensaje del tag: `"Guía 7: CRUD completo con formularios"`

Mensaje de commit final: `"feat: CRUD completo para libros y autores con formularios"`

— Fin de la Guía 7 —

INF560 — Desarrollo Web Backend • UATF Potosí